Gestion des fichiers séquentiels en C

Arfaoui Ahlem

Introduction (1):

- En C, les communications d'un programme avec son environnement se font par l'intermédiaire de fichiers.
- Pour le programmeur, tous les périphériques, même le clavier et l'écran, sont des fichiers.
- Jusqu'ici, nos programmes ont lu leurs données dans le fichier d'entrée standard, (c.-à-d.: le clavier) et ils ont écrit leurs résultats dans le fichier de sortie standard (c.-à-d.: l'écran).

  ➔ **Défaut d'utilisation des variables:**

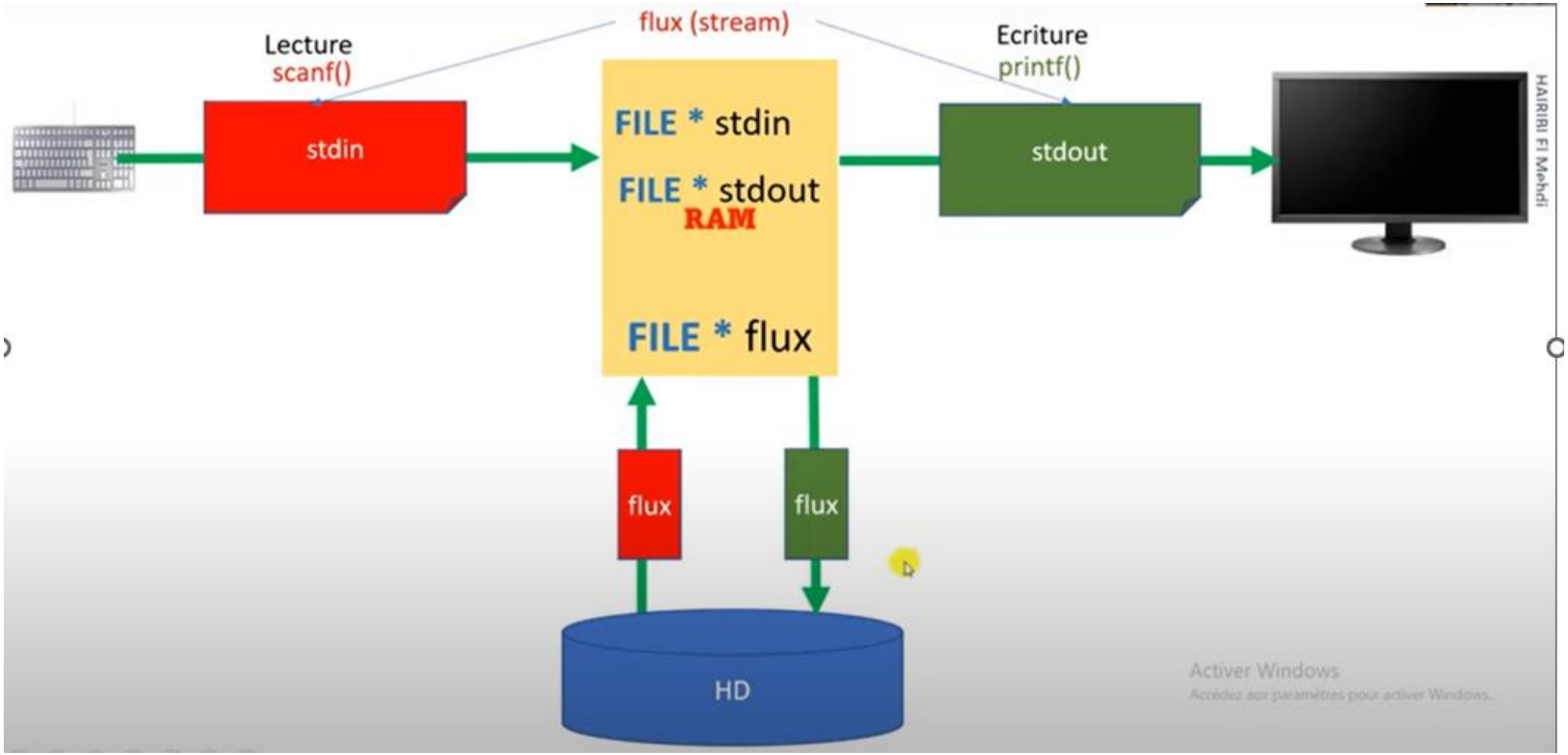
  ✓n'existent que dans la mémoire vive

  ✓il n'est pas possible de les récupérer une fois le programme est arrêté.

➔ Si on a à lire des données (texte, nombres,…) ou bien sauvegarder des résultats après l'arrêt du programme ?

  ✓ vous devrez les enregistrer dans un fichier.

Introduction (2):



Introduction (3):

- ✓ Un **fichier** est un ensemble structuré de données stocké en général sur un support externe (disquette, disque dur, disque optique, bande magnétique, ...).

- ✓ Un **fichier structuré** contient une suite **d'enregistrements** homogènes, qui regroupent le plus souvent plusieurs composantes appartenant ensemble **(champs)**.

- ✓ Dans **des fichiers séquentiels**, les enregistrements sont mémorisés consécutivement dans l'ordre de leur entrée et peuvent seulement être lus dans cet ordre.

- ➔ Si on a besoin d'un enregistrement précis dans un fichier séquentiel, il faut lire tous les enregistrements qui le précèdent, en commençant par le premier.

Introduction (4):

- On a **2 types de fichiers**:
  - ✓**Les fichiers textes:** constitués d'une suite de caractères (codés en ASCII)
  - ✓**Les fichiers binaires:** constitués d'une suite d'octets

- Lorsqu'on a des données numériques, il est souvent plus efficace de les écrire sous forme « binaire ».
- Le format binaire occupe beaucoup moins de place en mémoire que le format texte.

Accès aux fichiers séquentiels(1):

❑ Le type FILE*:

❖ Pour pouvoir travailler avec un fichier, un programme a besoin d'un certain nombre d'informations au sujet du fichier:

✓ Adresse de la mémoire,
✓ Position actuelle de la tête de lecture/écriture,
✓ Type d'accès au fichier: écriture, lecture, ...
✓ Etc.

❖ Ces informations sont rassemblées dans une structure du type spécial **FILE**. Lorsque nous ouvrons un fichier avec la commande **fopen**, le système génère automatiquement un bloc du type **FILE** et nous fournit son adresse.

Accès aux fichiers séquentiels(2):

❑ Le type FILE*:

❖ Tout ce que nous avons à faire dans notre programme est:

✓ Déclarer un pointeur du type **FILE*** pour chaque fichier dont nous avons besoin.

✓ Affecter l'adresse retournée par **fopen** à ce pointeur.

✓ Employer le pointeur à la place du nom du fichier dans toutes les instructions de lecture ou d'écriture.

✓ Libérer le pointeur à la fin du traitement à l'aide de **fclose**.

Accès aux fichiers séquentiels(3):

❑ Ouvrir un fichier séquentiel: fopen

❖ Lors de l'ouverture d'un fichier avec **fopen**, le système s'occupe d'établir une connexion entre la mémoire et le fichier.

FILE* fopen (const char * nomFichier, const char * modeAccès);

❖ **nomFichier :** est une chaîne de caractères constante qui représente le chemin du fichier.

✓ Seul si le fichier existe dans le même dossier que l'exécutable du programme c.

```
fichier=fopen("fichier.txt","a+");
```

✓ Avec un chemin relatif s'il existe dans un sous dossier du dossier du programme

```
fichier=fopen("dossier\fichier.txt","a+");
```

✓Avec un chemin complet si le fichier est placé ailleurs

```
fichier=fopen("c:\program\dossier\fichier.txt","a+");
```

Accès aux fichiers séquentiels(4):

❑ Ouvrir un fichier séquentiel: fopen

❖ Mode d'accès possible:

Mode	Type(s) d'opération(s)
w	Ecriture seule, si le fichier n'existe pas, il sera créé.
w+	Lecture et écriture, avec suppression du contenu au préalable, càd le fichier est d'abord vidé de son contenu, on pourra y écrire, et le lire ensuite. Si le fichier n'existe pas, il sera créé.
a	Mode d'ajout. On peut écrire à la fin du fichier. Si le fichier n'existe pas, il sera créé.
a+	Ajout en lecture / écriture à la fin. On peut lire et écrire du texte à partir de la fin du fichier. Si le fichier n'existe pas, il sera créé.
r	Lecture seule, le fichier doit avoir été créé au préalable.
r+	Lecture et écriture. Le fichier doit avoir été créé au préalable.

Pour un fichier binaire, on utilise ("rb", "wb", "ab", "rb+", "wb+", "ab+").

Accès aux fichiers séquentiels(5):

❑ **Ouvrir un fichier séquentiel: fopen**

❖ **Le résultat de fopen:**

✓ Si le fichier a pu être ouvert avec succès, **fopen** fournit l'adresse d'un nouveau bloc du type **FILE**.

✓ Si **fopen** retoune NULL ➔ Il y a une erreur.

❑ **Fermer un fichier séquentiel : fclose**

❖ **Le prototype:**

int fclose(FILE * fichier);

❖ Elle renvoie un entier qui indique si la fermeture est réussie ou non.

❖ Il vaut:

✓**0**: si la fermeture a marché

✓**EOF**: si la fermeture a échoué (EOF est un nombre négative)

➔Afin de libérer la mémoire, il faut toujours fermer le fichier après l'utilisation

Accès aux fichiers séquentiels(6):

Les **étapes à suivre** dans l'ordre pour gérer un fichier, que ce soit pour le lire ou pour y écrire:

1) Appeler la fonction d'ouverture de fichier **fopen** qui nous renvoie un pointeur sur le fichier.
2) Vérifier si l'ouverture a réussi en testant la valeur du pointeur qu'on a reçu.
    a) Si le pointeur vaut **NULL**, c'est que l'ouverture du fichier n'a pas fonctionné, dans ce cas on ne peut pas continuer (il faut afficher un message d'erreur).
    b) Si le pointeur est différent de **NULL**, c'est que l'ouverture a fonctionné, alors on peut lire ou écrire dans le fichier.
3) Une fois qu'on a terminé de travailler sur le fichier, il faut fermer le fichier avec la fonction **fclose**.

Accès aux fichiers séquentiels(7):

❑ Exemple:

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE * fp;
    fp=fopen("f2.txt","w");
    if(fp==NULL){
    printf("le fichier n'a pas pu etre ouvert\n");
    exit(1);
    }
    printf("le fichier exist\n");
    fclose(fp);

    return 0;
}
```

```
le fichier exist

Process returned 0 (0x0)   execution time : 0.012 s
Press any key to continue.
```

Ecriture dans des fichiers textes (1):

❖ Il existe **3 fonctions** capables d'écrire dans un fichier:

1) **fputc:** écrit un caractère dans le fichier (**UN SEUL** caractère à la fois).

Syntaxe:

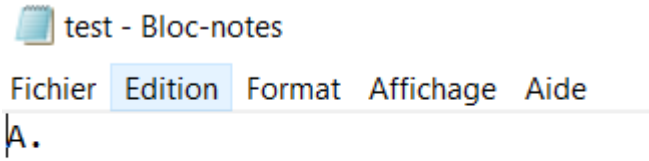
int fputc (int caractére, FILE * file);

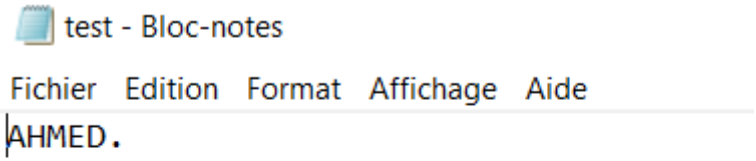
➔ Après avoir écrit un caractère dans le fichier texte, le pointeur de position interne est **incrémenté**.
➔ Si l'écriture réussit, **la valeur ASCII du caractère qui a été écrit est renvoyée**.
➔ En cas d'erreur, il retourne **EOF.**

Ecriture dans des fichiers textes (2):

Exemple:

```c
#include <stdio.h>
int main(void)
{     FILE *f;
    char ch;
    f = fopen("test.txt", "w");
    if(f==NULL){
        printf("Erreur lors de l'ouverture d'un fichier");
        exit(1);
    }
    do{
        printf("Donner un caractere, pour terminer (.)");
        fflush(stdin);
        scanf("%c",&ch);
        fputc(ch, f);
    }
    while(ch!='.');
    fclose(f);
    return 0;
}
```





Ecriture dans des fichiers textes (3):

2) **fputs:** écrit une chaînede caractère dans le fichier.

Syntaxe:

int fputs (const char * string, FILE * file);

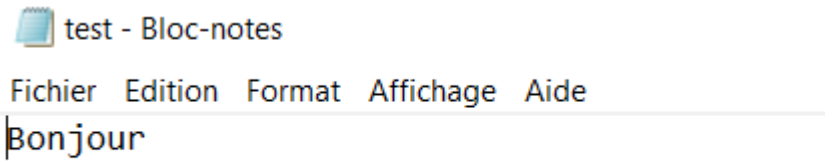
➔ Elle renvoie une valeur **non négative** en cas de succès, sinon **EOF** est renvoyé en cas d'erreur.

➔ Ecrit toute une chaîne dans un fichier

➔ Notons qu'avec **fputs**, la chaîne est considérée comme une constante, càd on écrit la chaîne sans la modifier

Exemple:

```c
#include <stdio.h>
int main(void)
{    FILE *f;
    char ch[20];
    f = fopen("test.txt", "w");
    if(f==NULL){
        printf("Erreur lors de l'ouverture d'un fichier");
        exit(1);}
        printf("Donner un caractere"); scanf("%s",ch);
        fputs(ch, f);
    fclose(f);
    return 0;}
```



Ecriture dans des fichiers textes (4):

3) **fprintf:** écrit une chaîne « formatée » dans le fichier .

Syntaxe:

int fprintf (FILE * file, const char * format,…);

➔ La fonction **fprintf()** est identique à **printf(),** mais au lieu d'écrire des données sur la console, elle écrit des données formatées dans le fichier. Presque tous les arguments de la fonction **fprintf()** sont identiques à ceux de la fonction **printf()**, à **la différence qu'il possède un argument supplémentaire qui est un pointeur de fichier** sur le fichier dans lequel la sortie formatée sera écrite.

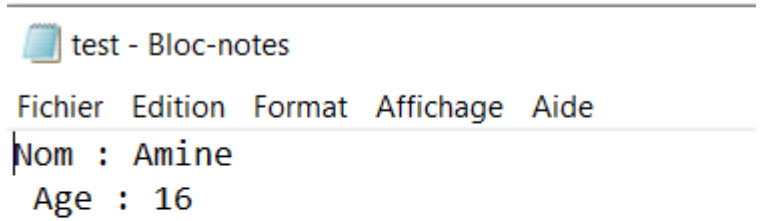
➔ En cas de succès, le nombre total de caractères écrits dans le fichier est renvoyé. En cas d'erreur, il retourne EOF.

Ecriture dans des fichiers textes (5):

Exemple:

```c
#include <stdio.h>
int main(void)
{
    FILE *f;
    char nom[20];
    int age;
    f = fopen("test.txt", "w");
    if(f==NULL){
        printf("Erreur lors de l'ouverture d'un fichier");
        exit(1);}
    printf("Saisir votre nom :");
    scanf("%s",nom);
    printf("saisir votre age");
    scanf("%d",&age);
    fprintf(f, "Nom : %s\n Age : %d\n", nom, age);
    fclose(f);

    return 0;
}
```



Lecture des fichiers textes (1):

❖ Il existe **3 fonctions** capables de lire un fichier:

1) **fgetc:** lire un caractère dans le fichier (**UN SEUL** caractère à la fois).

Syntaxe:

int fgetc (FILE * file);

➔ Cette fonction est complémentaire à la fonction **fputc().**

➔ Il lit **un seul caractère** du fichier et **incrémente le pointeur** de position du fichier. Si **fgetc()** est appelée **une seconde fois**, la fonction lira donc **le second caractère**, puis le troisième et ainsi de suite.

➔ Pour utiliser cette fonction, le fichier doit être ouvert **en mode lecture**.

➔ En cas de succès, il renvoie la valeur ASCII du caractère mais vous pouvez aussi attribuer le résultat à une variable de type char.

➔ En cas d'échec ou de **fin de fichier**, il renvoie **EOF.**

Lecture des fichiers textes (2):

Exemple:

```c
#include <stdio.h>
int main(void)
{
    FILE *f;
    char ch;
    f = fopen("test.txt", "r");
    if(f==NULL){
        printf("Erreur lors de l'ouverture d'un fichier");
        exit(1);
    }
    while((ch=fgetc(f))!=EOF){
        printf("%c", ch);
    }

    fclose(f);
    return 0;
}
```

```
Nom : Amine
 Age : 16

Process returned 0 (0x0)   execution time : 0.028 s
Press any key to continue.
```

Lecture des fichiers textes (3):

2) **fgets:** pour lire une chaîne de caractère dans le fichier.

Syntaxe:

char * fgets (char * string, int maxLength, FILE * file);

➔ Lit au maximum une ligne (elle s'arrête au premier \n qu'elle rencontre). Si vous voulez lire plusieurs lignes, il faudra faire une boucle.

➔ Prend en paramètre une chaine vide pour contenir ce qui est lu à partir du fichier, le nombre maximal de caractères à lire et un pointeur sur le fichier à lire.

➔ En cas de succès, elle retourne **un pointeur à la string**.

➔ En cas d'erreur ou de fin du fichier, elle renvoie **NULL**.

Lecture des fichiers textes (4):

Exemple 1:

```c
#include <stdio.h>
int main(void)
{
    FILE *f;
    char str[20];
    f = fopen("test.txt", "r");
    if(f==NULL){
        printf("Erreur lors de l'ouverture d'un fichier");
        exit(1);
    }
  if(f!= NULL){
  fgets(str,20,f) ;
        puts(str);
    }
    fclose(f);
    return 0;
}
```

```
Nom : Amine


Process returned 0 (0x0)   execution time : 0.031 s
Press any key to continue.
```

test - Bloc-notes

Fichier Edition Format Affichage Aide

```
Nom : Amine
 Age : 16
```

Lecture des fichiers textes (5):

Exemple 2:

```c
#include <stdio.h>
int main(void)
{
    FILE *f;
    char str[20];
    f = fopen("test.txt", "r");
    if(f==NULL){
        printf("Erreur lors de l'ouverture d'un fichier");
        exit(1);
    }
    while( fgets(str,20,f) != NULL){
        puts(str);
    }
    fclose(f);
    return 0;
}
```

```
Nom : Amine

 Age : 16


Process returned 0 (0x0)   execution time : 0.009 s
Press any key to continue.
```

Lecture dans des fichiers textes(6):

3) **fscanf:** pour lire une chaîne de caractère « formatée » dans le fichier.

Syntaxe:

int fscanf (FILE * file, const char * format,…);

➔ La fonction **fscanf()** est utilisée pour lire une entrée formatée à partir du fichier. Cela fonctionne comme la fonction **scanf()** mais au lieu de lire les données de l'entrée standard, il lit les données du fichier. En fait, la plupart des arguments de la fonction **fscanf()** sont identiques à ceux de la fonction **scanf(),** à la différence qu'il a simplement besoin d'un argument supplémentaire, bien évidemment **d'un pointeur de fichier.**

➔ En cas de succès, cette fonction renvoie **le nombre de valeurs lues** et en cas d'erreur ou de fin du fichier, elle renvoie **EOF.**

Lecture dans des fichiers textes (7):

Exemple:

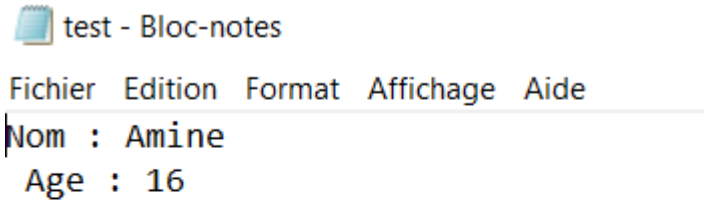
```c
#include <stdio.h>
int main(void)
{
    FILE *f;
    char nom[20];
    int age;

    f = fopen("test.txt", "r");

    if(f==NULL){
        printf("Erreur lors de l'ouverture d'un fichier");
        exit(1);
    }
    // lire les données à partie de fichier
    fscanf(f, "Nom : %s\n Age : %d\n", nom, &age);
    // afficher les données chargées
    printf("Nom : %s\n Age : %d\n", nom, age);
    fclose(f);
    return 0;
}
```

```
Nom : Amine
 Age : 16

Process returned 0 (0x0)   execution time : 0.033 s
Press any key to continue.
```



Ecriture dans des fichiers binaires(1):

- Dans cette partie, nous allons apprendre à lire et à écrire des données dans et à partir du fichier en mode binaire.
- Rappelez-vous qu'en mode binaire, les données sont stockées dans le fichier de la même manière que dans la mémoire, de sorte qu'aucune transformation de données n'a lieu en mode binaire.
- La fonction **fwrite**: permet d'écrire tout un bloc de données en binaire

Syntaxe:

int fwrite(void * source, int taille_type, int nombre, FILE *fp);

La fonction prend **quatre arguments**:

- ✓ Adresse des données à écrire sur le fichier (**source**)
- ✓ Taille des données à écrire sur le fichier (**taille_type**)
- ✓ Nombre de ce type de données(**nombre**)
- ✓ Pointeur vers le fichier où vous voulez écrire (**fp**).

Ecriture dans des fichiers binaires(2):

Exemple1: Ecrire une variable

```c
#include <stdio.h>
int main(void)
{
    FILE *f;
    int a=5;

    f = fopen("test.bin", "wb");

    if(f==NULL){
        printf("Erreur lors de l'ouverture d'un fichier");
        exit(1);
    }
    fwrite(&a,sizeof(a),1,f);
    fclose(f);
    return 0;
}
```

Ecriture dans des fichiers binaires(3):

Exemple2: Ecrire un tableau

```c
#include <stdio.h>
int main(void)
{
    FILE *f;
    int tab[4]={2,5,7,8};


    f = fopen("test.bin", "wb");

    if(f==NULL){
        printf("Erreur lors de l'ouverture d'un fichier");
        exit(1);
    }
    fwrite(tab,sizeof(tab),1,f);
    fclose(f);
    return 0;
}
```

Ecriture dans des fichiers binaires(3):

Exemple3: Ecrire une structure

```c
#include <stdio.h>
struct etudiant{
    char nom[20];
    int age;
};



int main(void)
{
    FILE *f;
    struct etudiant etd={"mostafa",34};
    f = fopen("test.bin", "wb");
    if(f==NULL){
        printf("Erreur lors de l'ouverture d'un fichier");
        exit(1);
    }
    fwrite(&etd,sizeof(etd),1,f);
    fclose(f);
    return 0;
}
```

Lecture dans des fichiers binaires(1):

- La fonction **fread()**: est complémentaire de la fonction **fwrite()**.
- La fonction **fread()** est couramment utilisée pour lire des données binaires.
- elle accepte les mêmes arguments que la fonction **fwrite().**

Syntaxe:

int fread(void * destination, int taille_type, int nombre, FILE *fp);

Exemple1: lire une variable

```c
#include <stdio.h>
int main(void)
{
    FILE *f;
    int a=5;
    f = fopen("test.bin", "rb");
    if(f==NULL){
        printf("Erreur lors de l'ouverture d'un fichier");
        exit(1);
    }
    fread(&a,sizeof(a),1,f);
    printf("%d",a);
    fclose(f);
    return 0;
}
```

```
5
Process returned 0 (0x0)   execution time : 0.025 s
Press any key to continue.
```

Lecture dans des fichiers binaires(3):

Exemple2: Lire une structure

```c
#include <stdio.h>
struct etudiant{
    char nom[20];
    int age;
};
int main(void)
{
    FILE *f;
    struct etudiant etd;
    f = fopen("test.bin", "r");

   if(f==NULL){
        printf("Erreur lors de l'ouverture d'un fichier");
        exit(1);
    }
    while( fread(&etd, sizeof(etd), 1, f) == 1 ){
        printf("Prénom : %s \n",etd.nom);
        printf("age : %d",etd.age);

        // vider le tempon mémoire
        fflush(stdin);
    }

    fclose(f);
    return 0;
}
```

```
Prenom : mostafa
age : 34
Process returned 0 (0x0)   execution time : 0.023 s
Press any key to continue.
```

Déplacer dans le fichier(1):

❖ A l'ouverture d'un fichier, il existe **un curseur virtuel** indiquant notre position dans le fichier.
➔ ce qui permet d'aller lire et écrire à une position précise dans le fichier.

❖ Il existe **trois fonctions** permettant de se déplacer dans un fichier:

✓ **ftell:** indique notre position actuelle dans le fichier .

Syntaxe:

long ftell (FILE * file);

✓ **rewind:** remet le curseur au début du fichier.

Syntaxe:

Void rewind(FILE *file);

✓ **fseek:** positionne le curseur d'un certain nombre de caractères indiqué par deplacement à partir de la position indiquée par origine.

- Deplacement: peut être positif (en avant), nul (= 0) ou négatif (en arrière).
- Origine peut prendre comme valeur l'une des trois constantes:

o **SEEK_SET** : indique le début du fichier
o **SEEK_CUR** : indique la position actuelle du curseur
o **SEEK_END** : indique la fin du fichier

Déplacer dans le fichier(2):

✓ **fseek:**

Syntaxe:

int fseek (FILE *file, long deplacement, int origine);

Exemple1: Placer le curseur deux caractères après le début

fseek (fichier,2, SEEK_SET);

Exemple2: Placer le curseur quatre caractères avant la position courante

fseek (fichier,-4, SEEK_CUR**);**

Exemple3: Placer le curseur à la fin du fichier

fseek (fichier,0, SEEK_END**);**

➔ **Remarques:**

- ✓ Si on écrit dans un fichier après avoir fait un **fseek** menant à la fin du fichier: **rajout à la fin du fichier.**
- ✓ Si on mène le curseur au début du fichier et on écrit: **écrire en écrasant le contenu ancien du fichier.**

Autres actions sur un fichier(1):

Renommer un fichier:

int **rename** (const char * ancienNom, const char * nouveauNom);

Supprimer un fichier:

int **remove** (const char * fichierAsupprimer);

Exercice 1:

Ecrire un programme qui crée sur disquette un fichier INFBIS.TXT qui est la copie exacte (enregistrement par enregistrement) du fichier INFORM.TXT.

Exercice 2:

Ajouter un nouvel enregistrement (entré au clavier) à la fin de INFORM.TXT et sauver le nouveau fichier sous le nom INFBIS.TXT.

Exercice 3:

Supprimer dans INFORM.TXT tous les enregistrements:
a) dont le numéro de matricule se termine par 8
b) dont le prénom est "Paul" (utiliser **strcmp**)
c) dont le nom est un palindrome. Définir une fonction d'aide PALI qui fournit le résultat 1 si la chaîne transmise comme paramètre est un palindrome, sinon la valeur zéro.
Sauver le nouveau fichier à chaque fois sous le nom INFBIS.TXT.